

ECE 385

Spring 2025

Final Project

Final Project: Chess

Russel De Leon, Vayun Gupta

Section: 12 pm Friday

Shixin Cheng

1. Introduction

For this project, we implemented chess on the Xilinx Spartan-7 xc7s50csga324-1 FPGA as a System on Chip (SoC). To do this, we used a MicroBlaze CPU for interfacing with the USB Mouse and VGA Monitor. Core components such as piece rendering, cursor display, and game logic are integrated through hardware and software co-design.

By offloading tasks like pixel-level rendering and sprite management to custom hardware modules on the FPGA, we achieved low-latency display updates with minimal CPU overhead. This hardware parallelism enables efficient real-time visual output without relying on a high-performance processor or operating system. Additionally, the modular design allows for an easily scalable project for future development.

To implement this functionality, we created a custom IP Block that interfaces with MicroBlaze through the AXI4-Lite Interface (as in lab 7.1) as well as built off of what we learned from lab 6.2 to render a cursor on top of the chessboard. Thus, we can broadly divide our design into the hardware design in Vivado, and the software design in Vitis.

2. Hardware design in Vivado

a) Written Description

i) Description of How I/O Works (6.1)

In Lab 6.1, the input/output (I/O) components—like switches, buttons, and LEDs—are connected to the FPGA through GPIO (General Purpose Input/Output) IP blocks. These GPIO IP blocks act as interfaces between the MicroBlaze processor and the physical hardware components.

The MicroBlaze processor accesses these I/O components using a memory-mapped interface, meaning each GPIO is assigned a specific memory address. When the software running on MicroBlaze wants to read from a switch or turn on an LED, it simply reads from or writes to that memory address. This makes I/O control very intuitive from the software side, as it treats hardware interactions just like reading or writing to a variable in memory.

ii) Description of MicroBlaze Interactions

- The MicroBlaze processor needs to receive input from a USB keyboard. To do this, it uses a USB host controller - the MAX3421E chip.
- The MicroBlaze communicates with the MAX3421E chip using the SPI (Serial Peripheral Interface) protocol. This SPI communication is handled by a hardware

module called AXI Quad SPI, which acts as a bridge between MicroBlaze and the MAX3421E.

- When you press a key on the keyboard, the MAX3421E detects it and sends the keycode (a number representing the key) to MicroBlaze.
- Once MicroBlaze receives the keycode, it passes this information to the ball module in ball.sv

iii) Description of VGA operation & Module Interactions

- 1) The VGA controller draws each pixel by going through all screen positions and managing timing
- 2) The Ball module tracks where the ball currently is
- 3) The Color Mapper uses this info to decide the color of each pixel (ball color or background)
- 4) Together, they create a live visual of a ball that moves in real time based on user input.

b) Algorithm/Color Mapper Explanation

We determine the RGB output for each pixel based on its position relative to a centered 480×480 chessboard and the current game state.

For each pixel (DrawX, DrawY), it first checks if the pixel lies within the bounds of the board. If so, it calculates local coordinates (localX, localY) and a linear ROM address to fetch the background pixel data from chessBoard_rom. The board is divided into 8×8 squares, so the square location is determined by dividing the local coordinates by the square dimensions.

Next, we use the square's file and rank to extract a 4-bit code from the chess codes input, which encodes the piece type (bits [2:0]) and color (bit [3]). Based on the piece type, the corresponding sprite ROM and palette are selected, and the pixel address within the sprite is computed using the modulus of the square dimensions.

The palette maps sprite pixel indices to 12-bit RGB values. If the piece is black (bit [3] = 1), the color is inverted unless it is red (to avoid the blue background bug as explained later).

We also compute the Euclidean distance from the pixel to a cursor center point and activate a circular cursor overlay if within radius.

On each positive VGA clock edge, we set the final RGB output based on priority: cursor > piece > board background > black (out-of-bounds). If a red color is encountered and the piece is a king

under checkmate, the red color is preserved (creating a highlight effect for the checkmated king); otherwise, it defaults to the board background color.

c) Module Descriptions

i) USB HDMI Top Level

Module:	mb_usb_hdmi_top.sv
Inputs:	logic [0:0] gpio_usb_int_tri_i, logic Clk, reset_rtl_0, usb_spi_miso, uart_rtl_0_rxd,
Outputs:	logic [2:0] hdmi_tmnds_data_n, hdmi_tmnds_data_p, logic gpio_usb_rst_tri_o, usb_spi_mosi, usb_spi_sclk, usb_spi_ss, uart_rtl_0_txd, hdmi_tmnds_clk_n, hdmi_tmnds_clk_p,
Description:	This module is the top level design that integrates all other modules together. Within the module the block diagram is instantiated.
Purpose:	This module is used to integrate USB mouse input, HDMI video generation, and UART communication.
Changes:	There were no changes made to this module.

ii) Hdmi Text Controller Top Level

Module:	hdmi_text_controller_v1_0.sv
Inputs:	logic [31:0] axi_wdata, axi_rdata, keycode0, chesscode0, chesscode1, chesscode2, chesscode3, chesscode4, chesscode5, chesscode6, chesscode7 logic [15:0] axi_araddr, axi_awaddr logic [3:0] axi_wstrb logic [2:0] axi_arprot, axi_awprot logic axi_aclk, axi_aresetn, axi_awvalid, axi_wvalid, axi_bready, axi_arvalid, axi_rready
Outputs:	logic [31:0] keycode1 logic [2:0] hdmi_tx_n, hdmi_tx_p, logic [1:0] axi_rresp, axi_bresp,

logic hdmi_clk_n, hdmi_clk_p, axi_awready, axi_wready, axi_bvalid,
axi_arready, axi_rvalid

Description: This module is the top level design that integrates all other modules together. Within the module, the AXI, Clock Wizard, VGA-To-HDMI, VGA controller, Mouse, and Chessboard Example are all instantiated within this module.

Purpose: This module is used to integrate USB mouse input, HDMI video generation, and UART communication.

Changes: The color mapper and font rom modules from lab 7.2 were removed while the chessboard_example module and mouse modules were instantiated.

iii) Hdmi Text Controller AXI

Module: hdmi_text_controller_v1_0_AXI.sv

Inputs: logic [31:0] S_AXI_WDATA,
 logic [15:0] S_AXI_AWADDR, S_AXI_ARADDR,
 logic [9:0] vram_index,
 logic [3:0] S_AXI_WSTRB,
 logic [2:0] S_AXI_AWPROT, S_AXI_ARPROT
 logic S_AXI_ACLK, S_AXI_ARESETN, S_AXI_AWVALID,
 S_AXI_WVALID, S_AXI_BREADY, S_AXI_ARVALID,
 S_AXI_RREADY,

Outputs: logic [31:0] S_AXI_RDATA, word_addr, controlRegAddr
 logic [1:0] S_AXI_BRESP, S_AXI_RRESP,
 logic S_AXI_AWREADY, S_AXI_WREADY, S_AXI_BVALID,
 S_AXI_ARREADY, S_AXI_RVALID,

Description: This module implements memory mapped register select, read and write logic generation, write response logic generation, axi_rready generation, axi_rvalid generation, etc. In addition to this, the module also instantates BRAM and provides the necessary output signals to be used for the Color Mapper.

Purpose: The purpose of the module is to help display text to the screen via HDMI by providing the necessary AXI read and write logic in addition to instantiating BRAM.

Changes: There were no changes made to this module.

iv) VGA Controller

Module: VGA_controller.sv

Inputs: logic pixel_clk, reset,

Outputs: logic [9:0] drawX, drawY,
logic hs, vs, active_nblank, sync

Description: This module generates the horizontal sync and vertical sync pulses in addition to the current pixel coordinates and a signal to indicate whether the current pixel is within the visible display area. The module utilizes horizontal and vertical line counters, following raster order.

Purpose: This module is used to define the display logic and synchronization signals in order to provide video output to a 640x480 resolution monitor.

Changes: There were no changes made to this module.

v) Mouse

Module: mouse.sv

Inputs: logic [31:0] keycode
logic Reset, frame_clk

Outputs: logic [9:0] BallX, BallY, BallS

Description: This module tracks and updates the position of a ball on the screen using USB mouse displacement data that is communicated from software to hardware. The 32-bit keycode input is parsed into button data and signed X/Y deltas (dx, dy). The ball's position is updated on each rising edge of frame_clk, unless a reset occurs. Additionally, the output position is clamped to the display bounds to ensure the ball remains visible and the ball size is fixed to 8.

Purpose: This module is used to simulate cursor movement on a screen using a ball.

Changes: N/A

vi) Chessboard

Module: Chessboard_example.sv
Inputs: logic [255:0] chesscodes, logic [9:0] BallX, BallY, DrawX, DrawY, Ball_size, DrawX, DrawY logic [7:0] button, logic vga_clk, blank, turn, isCheckmate
Outputs: logic [3:0] red, green, blue
Description: This module renders an 8x8 chessboard on screen by decoding a 256-bit chesscodes bus, which encodes piece type and color per square. The module also computes screen-aligned positions for each square, determines if a piece should be displayed at the current DrawX and DrawY pixel, and uses ROM-based sprites and palettes to render chess pieces. In addition to this, it also renders a cursor and highlights the checkmated king in red if isCheckmate is set. Piece colors are also inverted based on the side bit in sq_data in addition to transparency being set for each piece rendered on the board.
Purpose: This module handles all VGA rendering of the chessboard, including piece sprites, the cursor, and special visual states such as a checkmated king.
Changes: In order to account for rendering pieces on top of the chessboard, each piece's rom and color palette was instantiated within this module. Additionally, this module was modified so that the chessboard did not stretch across the entire screen as well as had additional inputs of DrawX, DrawY, button, isCheckmate, and chesscodes.

d) Block Design Components

- **Microblaze**

A soft processor core designed by Xilinx for FPGAs used to run embedded software on FPGA platforms.

- **Clocking Wizard**

Generates and manages clock signals for the system.

- **Microblaze Debug Module**

Enables debugging of the MicroBlaze processor using Vitis. Supports breakpoints, single-step execution, and memory inspection.

- **Processor System Reset**

Manages reset signals across the design. Ensures that all components start in a known state by synchronizing resets to the system clock.

- **Microblaze Local Memory**

On-chip memory, used for code or data storage during execution.

- **AXI Quad SPI**

An interface module that allows MicroBlaze to communicate with peripherals.

- **AXI TIMER**

A timer/counter module that can be used for timekeeping, delays, or triggering periodic events within the system.

- **AXI UARTLITE**

A lightweight UART interface for serial communication. Commonly used for console I/O or communicating with a PC via a serial terminal.

- **AXI Interconnect**

A bus matrix that connects master and slave AXI devices. Handles data routing and arbitration between MicroBlaze and peripherals.

- **AXI Interrupt Controller**

Manages multiple interrupt sources and prioritizes them for the MicroBlaze processor.

- **AXI GPIO (USB Reset)**

General-purpose I/O used to control USB reset functionality.

- **AXI GPIO (USB Interrupt)**

Monitors USB interrupt signals.

- **AXI GPIO (USB Keycode)**

Reads keycode data from the USB mouse

- **AXI GPIO (USB Chessboard0, USB Chessboard1, USB Chessboard2, USB Chessboard3)**

Reads data from the software for which piece is stored at the square, for each of the 64 squares of the chessboard. We have a 4-bit encoding for each square: 1 bit to denote the color, and 3 bits to denote the piece. Thus, we need 256 bits (or 4 32-bit dual channel GPIO modules) to represent the entire chessboard.

- **HDMI Text Controller**

Our customized IP that calculates what to display at the coordinates given by the software. For example, it chooses from the chess pieces, the chessboard, the mouse pointer, and the background black image. It also calculates the red highlight of the king when the king has been checkmated. Described in detail in the previous sections.

Block Diagram

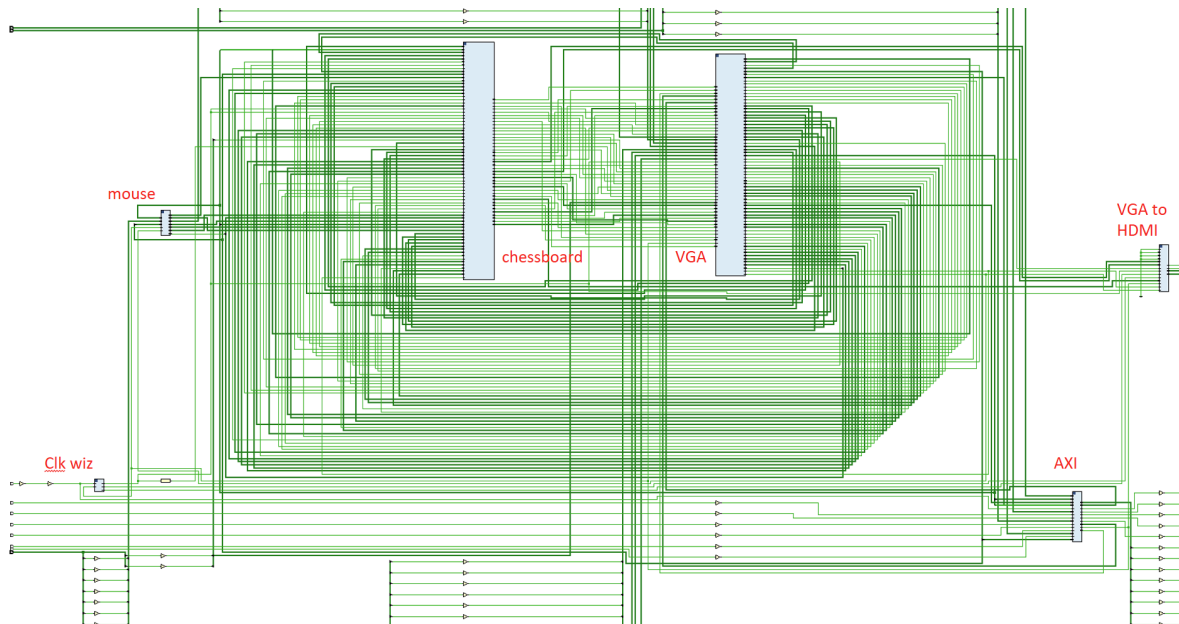


Figure 1: Image of RTL Block Diagram for SoC Chess

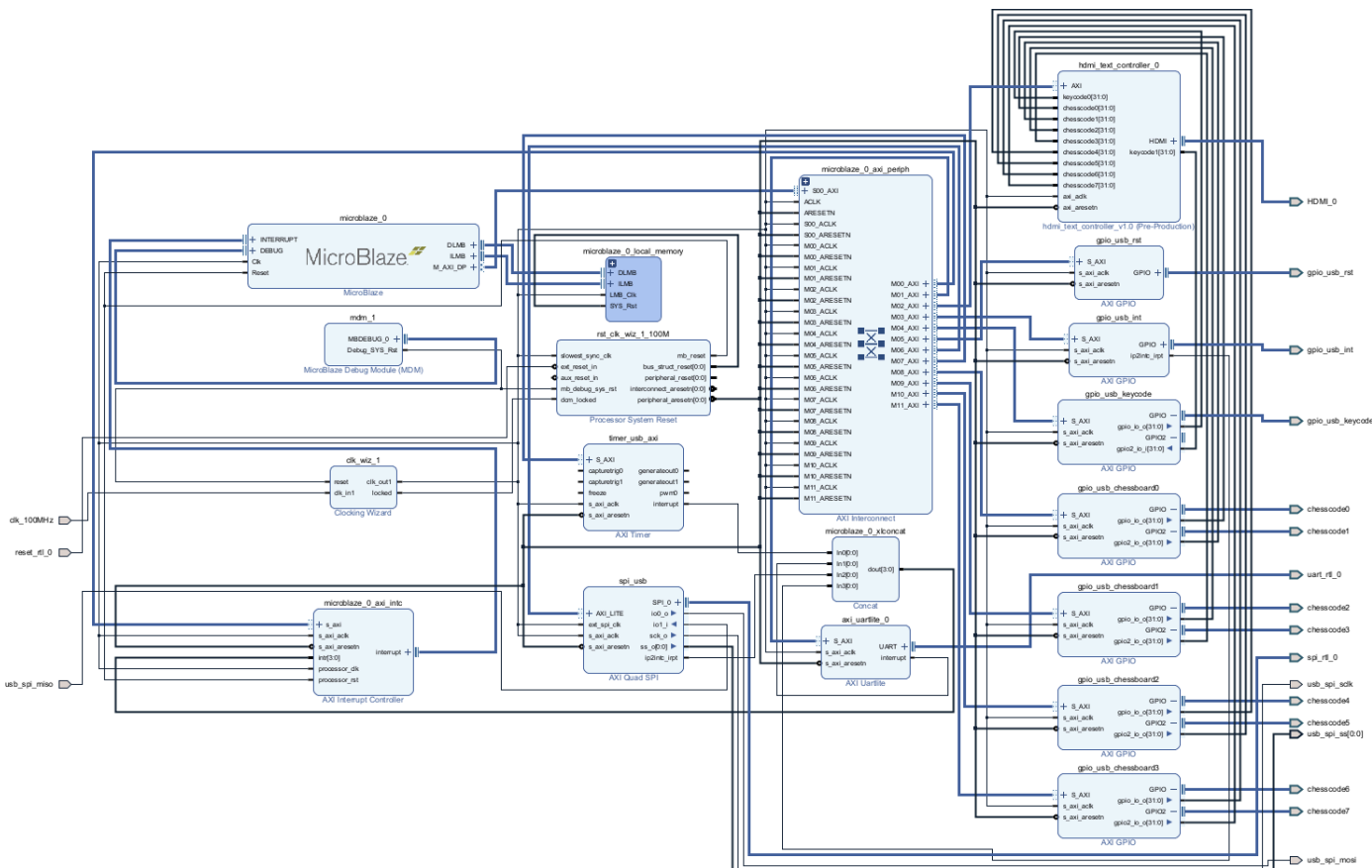


Figure 2: Image of Block Design

3. Software design in Vitis

a) Written Description of SPI Protocol & Operation

The SPI (Serial Peripheral Interface) protocol is a protocol for a master (eg. the Microblaze softcore processor on the Xilinx Spartan-7 FPGA) to communicate with one or more slaves (eg. the USB Keyboard).

It consists of:

- MOSI (Master Out, Slave In): Line for data sent from the master to the slave.
- MISO (Master In, Slave Out): Line for data sent from the slave to the master.
- SCLK (Serial Clock): Clock signal generated by the master to synchronize data transmission.
- SS/CS (Slave Select or Chip Select): Line used by the master to activate a specific slave device.

The master sets the clock speed and selects a slave device by pulling its CS/SS line low. Data is exchanged bit by bit: The master sends data on the MOSI line while the slave responds with data on the MISO line. After communication is complete, the master deactivates the CS/SS line.

b) Descriptions of C code Functions: Read & Write

i) AXI Read and Write

i) MAXreg_wr

Function: void MAXreg_wr_(BYTE reg, BYTE val)

Inputs: BYTE reg, val

Outputs: N/A

Description: This function performs a single host register write. This is done by first selecting the MAX3421E as the slave and writing reg + 2 via SPI as well as val via SPI followed by deselecting the MAX3421E.

Purpose: This module is used to perform a single host register write.

Changes: Within this function, we added the logic that would select and deselect the MAX3421E in addition to creating the necessary variables to properly utilize the XSpi_Transfer function. In addition to this, we also added the necessary print statements to read the return code and print an error.

ii) MAXbytes_wr

Function: BYTE* MAXbytes_wr(BYTE reg, BYTE nbytes, BYTE* data)

Inputs: BYTE reg, nbytes
BYTE* data

Outputs: BYTE* (data + nbytes)

Description: This function performs a multiple byte write and returns a pointer to the memory position after the last written byte. This is done by first selecting the MAX3421E as the slave, followed by writing reg + 2 via SPI as well as data[n] via SPI where n ranges from 0 to nbytes - 1. In addition to this, the function then reads the return code and prints an error while also deselecting the MAX3421E and returning data + nbytes.

Purpose: This module is used to perform a multiple byte write

Changes: Within this function, we added the logic that would select and deselect the MAX3421E in addition to creating the necessary variables to properly utilize the XSpi_Transfer function. In addition to this, we also added the necessary print statements to read the return code and print an error.

iii) MAXreg_rd

Function: BYTE MAXreg_rd(BYTE reg)

Inputs: BYTE reg

Outputs: BYTE val

Description: This function performs a single host register read. This is done by first selecting the MAX3421E as the slave and writing reg via SPI. Following this, the val is then read via SPI in addition to the return code, printing any errors if needed, and deselecting the MAX3421E.

Purpose: This module is used to perform a single host register read.

Changes: Within this function, we added the logic that would select and deselect the MAX3421E in addition to creating the necessary variables to properly utilize the XSpi_Transfer function. In addition to this, we also added the necessary print statements to read the return code and print an error.

iv) MAXbytes_rd

Function: BYTE* MAXbytes_rd(BYTE reg, BYTE nbytes, BYTE* data)

Inputs: BYTE reg, nbytes
BYTE* data

Outputs: BYTE* (data + nbytes)

Description: This function performs a multiple byte register read and returns a pointer to the memory position after the last read. This is done by first selecting the MAX3421E as the slave, followed by writing reg via SPI as well as reading data[n] via SPI where n ranges from 0 to nbytes - 1. In addition to this, the function then reads the return code and prints any errors if necessary while also deselecting the MAX3421E and returning data + nbytes.

Purpose: This module is used to perform a multiple byte write

Changes: Within this function, we added the logic that would select and deselect the MAX3421E in addition to creating the necessary variables to properly utilize the XSpi_Transfer function. In addition to this, we also added the necessary print statements to read the return code and print an error.

ii) Changes to USB Functions/Files

v) LW_USB Main.c

File: lw_usb_main.c

Description: This function initializes the hardware platform and USB stack to support interaction with USB HID devices such as a mouse or keyboard. It configures GPIO for displaying debug data and initializes a digital chessboard for

mouse-based gameplay. Within a continuous loop, the function handles USB state changes and polls connected devices (keyboard or mouse), parsing inputs and updating the game state accordingly. It also interprets mouse clicks as board selections and attempts to process chess moves, detecting checkmate or handling undo operations with right-clicks. USB state transitions are monitored and the system reinitializes if any errors occur

Purpose: This main control module integrates USB HID input with a digital chess application on an FPGA platform, managing USB initialization, device polling, and gameplay logic, while using GPIO for debug output and interaction feedback.

Changes: We added support for mouse click detection, calling the chess game logic when the mouse is clicked on valid chessboard coordinates.

vi) HID.c

File: HID.c

Description: This module implements support for USB Human Interface Devices (HID), specifically targeting boot protocol keyboards and mice. It includes functions to probe connected USB devices for HID descriptors, configure them into boot protocol mode, and poll them for input reports using low-level USB communication. The file also initializes and maintains endpoint data structures to facilitate interrupt-based data transfers from these devices.

Purpose: To provide low-level USB stack integration for boot protocol HID peripherals (mouse and keyboard).

Changes: We added support for our mouse, which is not a traditional USB Mouse. The receiver can receive a signal from the mouse wirelessly, and it is compatible with Keyboards and other devices too. Thus, it showed up as a USB Keyboard by the logic in HID.c earlier, and we had to modify the logic to also detect the Mouse input.

iii) Chess Logic

vii) Initialize GPIOs

Function:	void init_chess_gpios()
Inputs:	N/A
Outputs:	N/A
Description:	This function initializes all chessboard GPIOs through using their device ID, calling the XGpio_initialize function, and setting the data direction for the channels.
Purpose:	This module is used to initialize the chesscode gpios.
Changes:	N/A

viii) Send Chesscode

Function:	void send_chesscode_to_hw()
Inputs:	N/A
Outputs:	N/A
Description:	This function sends chesscodes from software to hardware. This is done by encoding each piece using a 4-bit code based on its piece type and color to create a 32-bit word. The word is then sent to the corresponding GPIO channel using the DiscreteWrite function where each rank is mapped to a specified GPIO module and channel.
Purpose:	This module is used to send the chesscodes back to hardware.
Changes:	N/A

ix) Initialize Board

Function:	void init_board()
------------------	-------------------

Inputs:	N/A
Outputs:	N/A
Description:	This function initializes the chessboard to the starting position. The global board array is initialized with the default piece layout using an 8x8 array of characters. Additionally, the initial positions for the white and black kings are set, the move history index is cleared, and the turn is reset to white.
Purpose:	This module is used to initialize the game board.
Changes:	N/A

x) Print Board

Function:	void print_board()
Inputs:	N/A
Outputs:	N/A
Description:	This function prints the current state of the chessboard to the console. Labels for both the rows and columns are printed using standard chess notation (files a-h and ranks 1-8). After displaying the board, the function then updates the hardware through calling send_chesscopde_to_hw().
Purpose:	This module is used to print the game board to the serial terminal.
Changes:	N/A

xi) File to Col & Rank to Row

Functions:	int file_to_col(char file), int rank_to_row(char rank)
Inputs:	char file, rank
Outputs:	tolower(file) - 'a', BOARD_SIZE - (rank - '0')

Description:	The file to col function converts a chess file character into its corresponding column index in the board array. The rank to row function converts a chess rank character into its corresponding row index in the board array.
Purpose:	These functions are used to convert user input in algebraic notation into array coordinates for board manipulation.
Changes:	N/A

xii) Apply Move

Function:	int apply_move(int row_from, int col_from, int row_to, int col_to)
Inputs:	int row_from, col_from, row_to, col_to
Outputs:	int 1 - successfully applied move
Description:	This function executes a move on the chessboard by transferring a piece from the source position to its destination. It then saves the current board state to the move history, while also handling pawn promotion to queen. The function also clears the source square and prints the updated board while also notifying the user via the terminal if the move resulted in a check.
Purpose:	This function is used to apply a move to the game board.
Changes:	N/A

xiii) Undo Move

Function:	void undo_move()
Inputs:	N/A
Outputs:	N/A
Description:	This function reverts the chessboard to the previous state by restoring the most recent entry in the move history. If at least one move has been made, it decrements the history index, copies the saved board state back into the current board, toggles the turn, and prints the restored board. If no moves have been made, it notifies the user that there are no moves to undo.

Purpose: This function is used to undo a move made to the game board.

Changes: N/A

xiv) Move Piece

Function: int move_piece(int col_from, int row_from, int col_to, int row_to) (e.g. move_pawn, move_rook, move_knight, move_bishop, move_queen, move_king)

Inputs: int col_from, row_from, col_to, row_to

Outputs: int 1 - valid normal move or valid capture
int 2 - valid en passant capture
int 0 - invalid move

Description: These functions validate legal movement for each chess piece based on the standard rules. Each function verifies that the intended move is consistent with the piece's movement pattern and does not violate board constraints such as capturing one's own piece or moving other pieces when not allowed. Special logic is included for:

- Pawns: advancing 1 or 2 squares, diagonal captures, and en passant
- Rooks: straight-line movement with obstacle checks
- Knights: L-shaped jumps, ignoring obstacles
- Bishops: diagonal movement with obstacle checks
- Queens: hybrid of rook and bishop movement
- Kings: single-square moves in any direction

Purpose: These functions are used to make a move for the corresponding piece selected.

Changes: N/A

xv) Castle

Function: int castle(int col_from, int row_from, int col_to, int row_to)

Inputs: int col_from, row_from, col_to, row_to

Outputs: int 1 - successful castle

int 0 - conditions are not met

Description: This function performs King-side or Queen-side castling. It checks whether the move corresponds to King-side or Queen-side castling, verifies that the involved rook is in its starting position, and ensures that the squares between the king and rook are unoccupied. If all conditions are satisfied, the function updates the board state to reflect the castling move, saves the previous state in the history buffer, and prints the updated board.

Purpose: This function is used to check for King-side or Queen-side castling and performs the move.

Changes: N/A

xvi) Can Piece Attack Square

Functions: int can_piece_attack_square(int row_from, int col_from, int row_to, int col_to)

Inputs: int row_from, col_from, row_to, col_to

Outputs: int 1 - piece can legally attack the target square via a standard move or capture
int 2 - if a pawn can perform a valid en passant capture
int 0 - piece cannot attack the target square

Description: This function determines whether a piece located at a given board square can legally attack another square. It identifies the type of the piece and delegates logic to the corresponding piece-specific move function. The return value reflects whether the attack is valid.

Purpose: This function is used to check to see if a piece can attack a given square.

Changes: N/A

xvii) Is Check

Function: int is_check()

Inputs: N/A

Outputs: int 1 - if current player's king is in check (illegal move)

int 2 - if move places the opponent's king in check
int 0 - if neither king is in check

Description: This function determines whether the current player is placing the opponent's king in check or if their own king is in check (which would invalidate the move). It first determines the current player based on turn, then iterates over the board to test all pieces.

Purpose: This function is used to see if the user is in check as well as see if the opponent is in check.

Changes: N/A

xviii) Leaves My King in Check

Function: int leaves_my_own_king_in_check(int row_from, int col_from, int row_to, int col_to)

Inputs: int row_from, col_from, row_to, col_to

Outputs: int 1 - if the move leaves the current player's king in check
int 0 - otherwise

Description: This function simulates a move on the board to determine whether it would leave the moving player's own king in check or not. It temporarily performs the move, updates the king positions and checks for a resulting check using is_check(). The board and king state are then restored to their original values and the result reflects whether the simulated move would violate the rule against moving into check.

Purpose: This function is used to validate move legality by ensuring that no move results in the player's own king being in check.

Changes: N/A

xix) Has Any Legal Move

Function: int has_any_legal_move()

Inputs: N/A

Outputs: int 1 - if at least one legal move exists for the current player
int 0 - no legal moves are available

Description: This function checks whether or not the current player has any legal moves remaining. This is done by iterating through all pieces belonging to the current player and testing every possible destination square on the board. If a move does not leave the player's own king in check, the function returns 1. If no such move is found, the function returns 0.

Purpose: This function is used to detect if the current player has any valid moves remaining.

Changes: N/A

xx) In Check Current Side

Function: int in_check_current_side()

Inputs: N/A

Outputs: int 1 - if the current player's king is in check
int 0 - otherwise

Description: This function determines whether the current player is in check by temporarily flipping the turn variable and calling is_check(). Since is_check() checks if the opponent's king is in check, the inversion allows it to detect whether the current player's king is under attack. The original turn is then restored and the function returns 1 if the current player's king is being attacked and 0 otherwise.

Purpose: This function is used to check and see if the current user is in check.

Changes: N/A

xxi) Is Checkmate

Function: int is_checkmate()

Inputs: N/A

Outputs:	int 1 - if the current player is in checkmate int 0 - otherwise
Description:	This function determines whether the current player is in checkmate. This is done by first checking if the player's king is in check using <code>in_check_current_side()</code> . If the king is not in check, the function returns 0. If the king is in check, it then checks whether the player has any legal move available using <code>has_any_legal_move()</code> . If no legal move exists, the function returns 1, indicating checkmate.
Purpose:	This function is used to check for checkmate.
Changes:	N/A

xxii) Make Move

Function:	<code>int make_move(char from_rank, char from_file, char to_rank, char to_file)</code>
Inputs:	<code>char from_rank, from_file, to_rank, to_file</code>
Outputs:	int 1 - if the move was successful int 0 - otherwise
Description:	This function converts a move written in algebraic notation into board indices and attempts to execute it. It performs several checks to determine whether the move is valid or not, including whether the piece can legally move or capture, whether castling is allowed, and whether the move would leave the king in check. If the move is valid, the move is committed using <code>apply_move</code> and the turn is changed accordingly.
Purpose:	This function is used to make a move to the game board.
Changes:	N/A

5. Simulations and Images

a) Annotated simulation of AXI write Transaction (7.1)

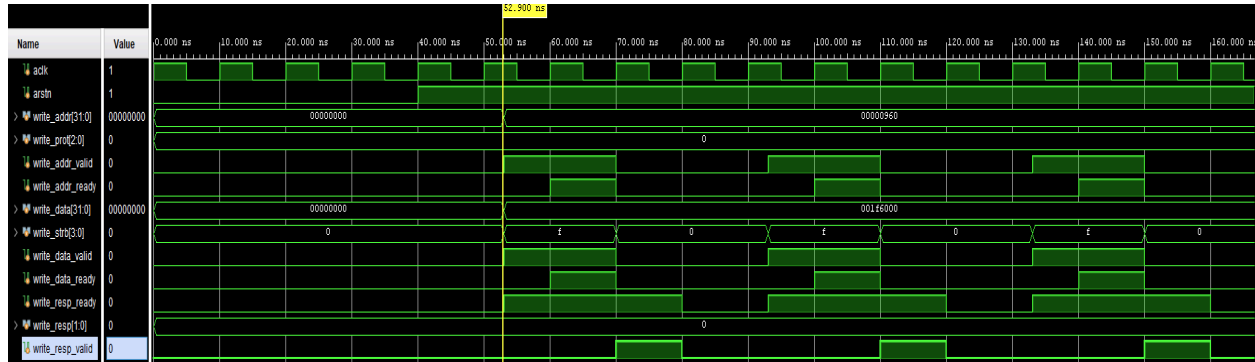


Figure 3: Image of AXI Write Transaction from 7.1

The above simulation shows AXI Write Operations:

- Master (Microblaze) puts Write Address on the write address channel
 - Asserts AWVALID (write address is now valid)
 - Asserts WVALID (write data is now valid)
 - Asserts BREADY (ready to receive data from the slave)
- Slave (HDMI IP) asserts AWREADY (slave is ready to receive address on the bus) and WREADY (slave is ready to receive write data on the bus).
- On the next clock cycle, master and slave de-assert AWVALID and WVALID, AWREADY and WREADY respectively. Slave simultaneously receives the write data and write address.
- The slave asserts BVALID (data on the write response channel is valid). In this case, the response is: 2'b00 ('OKAY').
- On the next clock cycle, the transaction ends with both BREADY and BVALID being high.

If all 32 bits in the register are not needed to be written, we use a Write Strobe.

b) Annotated simulation of AXI read Transaction (7.1)

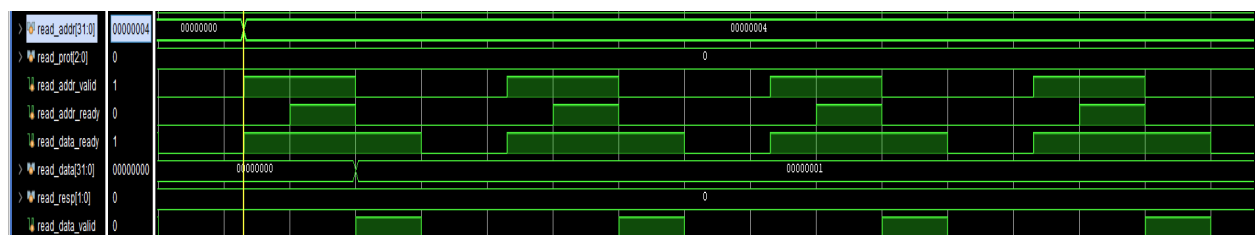


Figure 4: Image of AXI Read Transaction from 7.1

The above simulation shows AXI Read Operations:

- Master puts Read Address on the read address channel
 - Asserts ARVALID (address is now valid)
 - Asserts RREADY (master is ready to receive data from the slave)
- Slave asserts ARREADY (slave is ready to receive address on the bus)
- On the next clock cycle, master and slave de-assert ARVALID and the ARREADY, respectively. Slave simultaneously receives the read address
- The slave puts the requested data on the Read Data channel and asserts RVALID (data on read channel is valid)
- On the next clock cycle, master and slave de-assert RREADY and the RVALID, respectively. Master simultaneously receives the read data

6. Analysis

Hardware

Below are some of the metrics for this lab:

$$WNS = 1.561 \text{ ns}, T_{clk} = 10 \text{ ns}$$

$$F_{max} = \frac{1}{T_{clk} - WNS}$$

$$F_{max} = \frac{1}{(10 - 1.561) \text{ ns}} = 118.50 \text{ MHz}$$

Table 1: Metrics for the Chess SOC Hardware

	Multiplier
LUT Usage	908
DSP Usage	8
Memory (BRAM)	39
Flip-Flop Usage	537
Latches	0
Frequency	118.50 MHz
Static Power	0.075W
Dynamic Power	0.398W
Total Power	0.473W

Software

Since all operations in the chess game logic do not involve anything more computationally intensive than going through each of the 64 squares of the chessboard (for example, in the ‘leaves my king in check’ function), each operation executes in constant time. Thus, all of the chess game logic is $O(1)$, and should not take a noticeable amount of time to execute.

7. Conclusion

Discuss functionality of your design. If parts of your design did not work, discuss what could be done to fix it.

Initially, when adding chess sprites to our design we ran into a bug where the background for the black pieces had a blue square around them. In order to fix this, we created an `out_r`, `out_g`, `out_b` variable, separate from the palette ones. We then initialized these variables to white - palette through checking to ensure that the square data's most significant bit indicated an inverse (1 indicated black piece, 0 indicated white) and the palette's rgb wasn't red. Otherwise, `out_rgb` would be set to the palette's rgb. In addition to this, we also had a bug where the white side pieces and blackside pieces were flipped (white side appeared on top of screen while black side appeared on bottom). In order to fix this, we carefully reviewed our VGA output and ensured that our logic was correct for checking for a red background (the color we used to be able to make pieces transparent). We also had a few bugs when it came to obstacle detection for piece movement such as white side rook being able to move from its starting position to black side starting rook position, ignoring the pawns in its path. These issues were fixed through extensive testing and reviewing each piece's functions accordingly.



Figure 5: Image of blue background for black pieces

What are some potential extensions of this design?

- Some potential extensions of this design would be to incorporate the original Font Rom from lab 7.2 to support IBM text and be able to display whose turn it is. Additionally, some further extensions would be to display captured pieces for both sides, add a start and end screen, as well as incorporate multiple modes such as Player vs A.I. and maybe even just a free play mode. Furthermore, some potential extensions would also include animated sprites for celebrations, adding sound for moves and captures, as well as adding a timer that pauses after a user makes a move.